

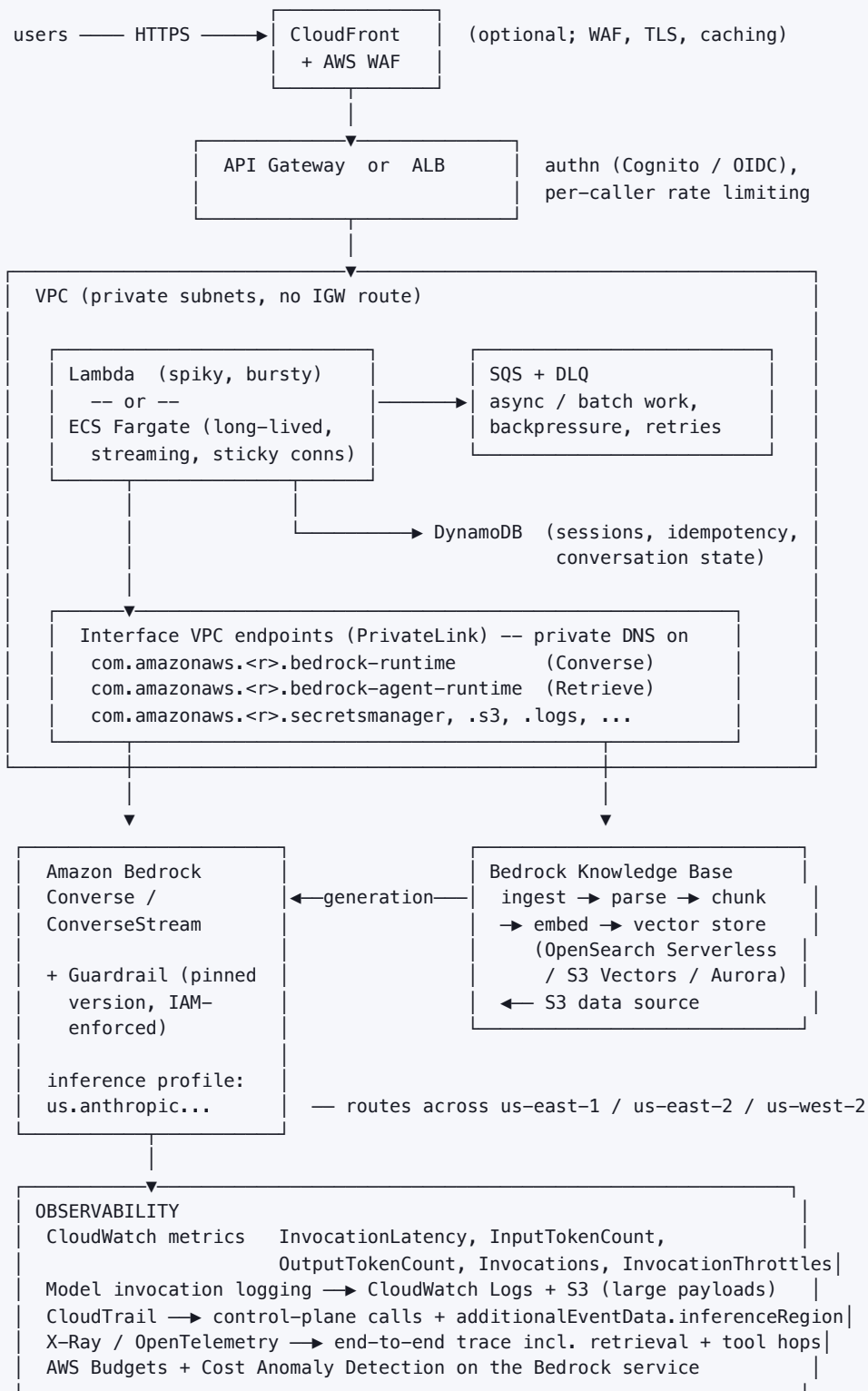
Production Craft — Cost, Quotas, Security and the Architecture

Track 16.6 –  AI on AWS with Amazon Bedrock · 16-ai-tech-stack/tracks/aws-bedrock/notes/02-production-craft.md

Note 01 was the API. This one is the part that decides whether your Bedrock application survives its first month in production.

1. Reference architecture

A production RAG + agent application on AWS. Every box here exists because something broke without it.



Lambda or ECS? Lambda for request/response and bursty traffic — it scales to zero and the per-invocation model matches per-request LLM cost. ECS Fargate once you stream to browsers, hold long-lived connections, or run agent loops that outlive Lambda's 15-minute ceiling. Do not stream a five-minute agent trace through Lambda and a NAT Gateway; you will meet the 350-second idle timeout.

IaC note. All of this is Terraform/CDK-able except the parts that are not. The `aws_bedrockagent_knowledge_base` and `aws_bedrockagent_data_source` resources exist in the AWS provider, as do `aws_bedrock_guardrail` and `aws_bedrock_model_invocation_logging_configuration`. What routinely is *not* covered on day one of a new feature: the vector index inside OpenSearch Serverless (you need the `opensearch` provider or a bootstrap Lambda), the Anthropic use-case form, and whichever preview feature shipped last week. Expect a thin custom-resource layer. Version-pin the provider, put the guardrail definition in the repo, and treat the guardrail **version number** as a deployable artifact — a guardrail edited in the console is an untracked production change.

2. Cost engineering

The three throughput models

Mode	Billing	Latency	Use it for
On-demand	Per token	Interactive	Everything, until you have data
Provisioned Throughput	Per Model Unit per hour	Interactive, guaranteed	Sustained predictable load; required for custom models
Batch inference	Discounted per token	Hours	Offline scoring, backfills, bulk classification

On-demand is the right default and most teams should never leave it. Provisioned Throughput is billed hourly whether you use it or not, with commitment terms of none / 1 month / 6 months (longer commitment, lower hourly rate) — and you cannot delete a committed PT before its term ends. A Model Unit delivers a specific tokens-per-minute capacity that AWS will not publish; you have to ask your account team. Buying PT to fix throttling you have not characterised is how teams end up with a six-month commitment for traffic that turned out to be a retry storm.

PT also **does not work with inference profiles** — buying it means giving up cross-Region routing for that model. That is a real architectural trade, not a footnote.

Batch inference takes JSONL on S3 in either `InvokeModel` or `Converse` format, runs `CreateModelInvocationJob`, and writes results back to S3 at a discount. Two hard limits: **no tool calling** and **no structured output** (each record is processed independently), and **no prompt caching**. Batch is for "score 4 million support tickets", not for "run my agent cheaply".

There is also a **service tier** dimension on the request itself: `"serviceTier": {"type": "reserved" | "priority" | "default" | "flex"}`. `flex` trades latency variance for cost on work that can wait. Read the service-tiers page before you set it.

Token accounting, and the trap

```
totalTokens = inputTokens + outputTokens
```

...until you turn on prompt caching, at which point the docs are explicit: `inputTokens` counts **only the non-cached** tokens. The real figure is:

```
true input = inputTokens + cacheReadInputTokens + cacheWriteInputTokens
```

The dashboard trap. Build your token dashboard on `inputTokens` alone and the day you enable caching it will show a 60% drop in "usage". Someone will believe it and plan capacity around it.

Prompt caching

A `cachePoint` block ends a cacheable prefix:

```
system=[{"text": "<1,024+ tokens of stable instructions>"},  
        {"cachePoint": {"type": "default"}}]      # or {"type": "default", "ttl": "1h"}
```

The mechanics that decide whether it works:

- **Checkpoints are evaluated** `tools` -> `system` -> `messages`, and the minimum token count is **cumulative across all three**, not per section. Changing a tool definition invalidates the system and message caches downstream of it. Put stable content first; put the cache point after it.
- **Minimums are per model** — 512, 1,024 or 4,096 tokens depending on the model, typically up to 4 checkpoints. Below the minimum, the request **succeeds and silently does not cache**. Verify with `cacheReadInputTokens`.
- **TTL** is 5 minutes by default; some models support `"ttl": "1h"`, and a successful cache hit resets it. Longer-TTL entries must appear before shorter-TTL ones in the same request.
- There is also **implicit caching**, where Bedrock reuses eligible prefixes with no markup in your request. Best-effort — never assume a hit.
- **Not supported with batch inference.**

A worked estimate

The full arithmetic runs in `../code/bedrock_simulator.py` §7. Stated assumptions, invented rates:

```
50,000 requests/day; 2,000 input tokens (1,600 a stable prefix);  
400 output tokens; 85% cache-hit rate.  
ILLUSTRATIVE rates per 1M tokens: in $3.00, out $15.00,  
cache read $0.30, cache write $3.75.
```

```
no caching    $600.00/day    $219,000/yr  
with caching  $425.40/day    $155,271/yr    (-29%)
```

Those rates are made up. Real Bedrock pricing varies by model, Region, service tier and endpoint, and changes. Price from <https://aws.amazon.com/bedrock/pricing/> and from Cost Explorer, never from a tutorial — including this one. Use the structure, not the numbers.

The levers, in the order that actually pays

1. **Right-size the model.** Routing, extraction, classification and summarisation usually do not need your best model. Moving 70% of traffic to a Haiku-class model typically beats every other optimisation combined. Measure the quality delta on a real eval set (Module 15) before and after.
2. **Cache the stable prefix.** Free, large, and the win compounds with traffic.
3. **Bound `maxTokens`.** Output tokens are the expensive ones. `maxTokens` is a cost control, not just a safety net.
4. **Move what can wait to batch.** If tool use is not needed, the discount is real.
5. **Trim the prompt.** Retrieved context is usually where the bloat is — eight chunks where four would do, doubled.
6. **Only then** consider Provisioned Throughput, and only with a month of utilisation data in hand.

Attribution

Two mechanisms, use both:

- **`requestMetadata`** on the call — `{"team": "...", "environment": "...", "feature": "..."} —` recorded in model invocation logs and filterable.
- **Application inference profiles** — create one per team or workload and tag it; costs then show up in AWS cost allocation tags.

Then: AWS Budgets with an alert on the Bedrock service, and Cost Anomaly Detection. An agent with a broken loop can spend a month's budget overnight and the first signal will be the bill.

3. Quotas, throttling, and failover

Model inference quotas are **token-based** (some models also have RPM), per model, per Region, per endpoint. `bedrock-runtime` counts input and output together with a model-specific output burn-down rate; `bedrock-mantle` uses separate input and output TPM and no RPM at all. **A quota is a ceiling, not a guarantee** — you can be within quota and still get a capacity error.

Read the error correctly

Code	HTTP	Means	Retry?
ThrottlingException	429	You exceeded an account quota	Yes, backoff
ModelNotReadyException	429	Model warming	Yes
ServiceUnavailable	503	Service capacity, not your quota	Yes
overloaded_error	529	Model overloaded; honour Retry-After	Yes
ValidationException	400	Your request is wrong	No
AccessDeniedException	403	IAM or model access	No
ResourceNotFound	404	Bad model ID or wrong Region	No

Retrying a 400 or 403 does not fix it and does burn quota. This is how a bug becomes an outage.

Retries

Do not hand-roll it. boto3 already implements exponential backoff with full jitter — just don't accept the default, which is legacy:

```
from botocore.config import Config
cfg = Config(
    retries={"total_max_attempts": 6, "mode": "standard"}, # 1 call + 5 retries
    read_timeout=300, # longer than worst-case generation
    connect_timeout=10,
    tcp_keepalive=True, # the 350s idle-timeout fix
)
client = boto3.client("bedrock-runtime", config=cfg)
```

The delay formula is `random(0, 1) * min(20s, base * 2**attempt)`. **Full jitter** — random *between zero and* the backoff, not backoff plus noise — is what decorrelates a thundering herd. `standard` mode also carries a retry quota (a token bucket) so that sustained failure fails fast instead of queuing.

`read_timeout` deserves a sentence of its own: set it shorter than the model's worst-case generation time and you will time out mid-answer, retry, and pay twice for tokens you never saw.

Sustained capacity errors are a different problem

Retries amplify them. When 503s or 529s persist: stop ramping, return to the last stable request rate, bound client-side concurrency with a token-aware limiter (not an RPM limiter — request sizes vary wildly), shed low-priority work, and use cross-Region inference. Then consider PT.

Ramp gradually. On-demand capacity varies by model, Region and time of day. Launch at a known-stable baseline, hold each level long enough to read success rate, 429/503/529 counts, latency percentiles and token consumption, then step up one dimension at a time.

Multi-Region failover

Three tiers, in increasing order of effort:

1. **Cross-Region inference profile.** Free, no code, no routing surcharge. Covers most capacity blips. This is your baseline.
2. **Application-level Region failover.** A second client in another Region, tried after a bounded number of retries. Verify model availability in the target Region — availability differs — and apply bounded retries so failover does not itself become a surge.
3. **Model failover.** Fall back to a different model family when yours is unavailable. Only works if you built on Converse, and only if you have eval coverage for the fallback model. Otherwise you have swapped an outage for a silent quality regression, which is worse because nobody pages for it.

Latency

Latency-optimized inference exists (`"performanceConfig": {"latency": "optimized" | "standard"}`), but as of this writing it is a preview limited to a short list of models and Regions, and once you exhaust the optimized quota your request is served at standard latency and billed at standard rates. Check the current support table before you design around it.

The bigger latency lever for most applications is not a flag: **stream**. Time-to-first-token is what users perceive. A streamed response that takes eight seconds to complete feels faster than a buffered one that takes four.

4. Observability

Four layers, and most teams ship with one:

Layer	Gives you	Default
CloudWatch metrics	Invocations, InvocationLatency, InputTokenCount, OutputTokenCount, InvocationThrottles, per model	On
Model invocation logging	Full prompts and completions, token counts, <code>identity.arn</code> , <code>requestMetadata</code>	Off
CloudTrail	Control-plane calls; <code>additionalEventData.inferenceRegion</code> for CRIS	On (management events)
X-Ray / OpenTelemetry	End-to-end trace: HTTP -> retrieval -> tool hops -> generation	Your job

Turn on model invocation logging on day one. Without it you cannot answer "what did the model actually see?", which is the first question in every incident and every compliance review. Bodies up to 100 KB are inline; larger payloads and binaries go to the S3 location you configure in `largeDataDeliveryS3Config` — configure it or you lose long documents from the record. It captures `bedrock-runtime` only; `bedrock-mantle` calls are not logged.

Then set alarms on the things that actually fail:

- `InvocationThrottles` > 0 sustained — you are at a quota edge.
- p99 `InvocationLatency` — model-side, excludes your network.
- Token rate per minute against your known quota — the leading indicator.
- Agent hop count per session — a runaway loop shows here before it shows on the bill.
- Guardrail intervention rate — a spike is either an attack or a regression.

And one dashboard nobody builds until after the incident: **cost per completed task**, not cost per request. A cheaper model that needs three attempts is not cheaper.

5. Security posture in one page

Control	Do this
IAM	<code>bedrock:InvokeModel</code> + <code>bedrock:InvokeModelWithResponseStream</code> scoped to specific model and inference-profile ARNs. Never <code>bedrock:*</code> on <code>*</code> .
Cross-Region IAM	Grant the profile and the model in the source and every destination Region, or you get intermittent 403s
Guardrail enforcement	IAM condition on <code>bedrock:GuardrailIdentifier</code> , pinned to a version , never <code>DRAFT</code>
Model allowlist	SCP denying invocation of anything outside your reviewed list — with a <code>bedrock:InferenceProfileArn</code> carve-out so CRIS keeps working
Network	Interface VPC endpoints for <code>bedrock-runtime</code> and <code>bedrock-agent-runtime</code> ; private DNS on; endpoint policy attached
Encryption	TLS 1.2 minimum. CMKs on guardrails, knowledge bases, agents, evaluation jobs, and log destinations. FIPS endpoints where required
Audit	Model invocation logging on, retention set, log group encrypted and access-restricted
Residency	Geographic CRIS profile, not global, when you have an obligation. Write down the decision

Two things worth repeating because they are non-obvious:

- **Blocked content is logged in plain text.** A guardrail that blocks a prompt full of PII has just written that PII to your log group. Encrypt, restrict, set retention.
- **Guardrails do not inspect tool results.** With `toolConfig`, `toolResult` content, tool descriptions and generated tool arguments are not evaluated. Untrusted text entering through a tool bypasses your content policy entirely unless you call `ApplyGuardrail` on it yourself. This is the primary prompt-injection path in an agentic Bedrock application (Module 13 [13-ai-security]).

On the training question: prompts and completions are used to generate your response and are not used to train the base models, and model providers have no access to them. What you should actually verify for an audit is narrower and more useful — what abuse-detection storage applies to your chosen models, what your

CRIS geography means for where data may be processed, and what your own log retention policy is, since your logs are the part you control. Details in Module 14 [14-ai-compliance].

6. Evaluation

Bedrock ships evaluation as a first-class feature, and you should use it as a floor, not a ceiling:

- **Programmatic model evaluation** — built-in or custom prompt datasets.
- **Human evaluation** — your own workforce or subject-matter experts.
- **LLM-as-a-judge** — a second model scores responses with explanations.
- **RAG evaluation** — scores a knowledge base's retrieval and generation against ground-truth passages and answers.

The RAG evaluation is the one that earns its keep: it is the fastest way to answer "is chunking my bottleneck, or is it the generation prompt?" — which determines whether you should be tuning a Knowledge Base or replacing it.

But keep your own regression suite in your own repo. Managed eval is great for comparing candidates on a fixed dataset; it is not a substitute for a versioned suite of your real failure cases running in CI. That is Module 15, and it is not optional.

7. A migration checklist

Going from "it works in the console playground" to "it is in production":

```
[ ] modelId is an inference-profile ID, verified with list-inference-profiles
[ ] Anthropic use-case form submitted for the account / org
[ ] IAM scoped to specific ARNs, both Invoke actions, all CRIS destinations
[ ] SCP allowlist for models and Regions, with the CRIS carve-out, tested
[ ] botocore Config: standard retries, 6 attempts, read_timeout, tcp_keepalive
[ ] every stopReason handled, including max_tokens and guardrail_intervened
[ ] tool loop has a hop limit and returns status:"error" on failure
[ ] guardrail created, VERSIONED, enforced by IAM condition, thresholds measured
[ ] ApplyGuardrail called on tool results and retrieved documents
[ ] VPC endpoints for bedrock-runtime AND bedrock-agent-runtime
[ ] model invocation logging on, with largeDataDeliveryS3Config set
[ ] requestMetadata on every call; application inference profile tagged
[ ] AWS Budget + Cost Anomaly Detection on Bedrock
[ ] alarms on InvocationThrottles, p99 latency, token rate, hop count
[ ] eval suite in CI, with a baseline captured before the first model swap
[ ] a written answer to "what happens when us-east-1 is throttled?"
```

If you cannot tick the last line, you do not have a production system; you have a demo with a load balancer in front of it.